FEBAUTO Racing

Release 2: keeping your driver

For	anyone who wrote code in the old devkit's `stack/` folder
Time	about 30 minutes, 20 of them a download
Result	your driver running in the new box, ready to score and submit

Formula Electric at Berkeley
September 2026, platform release v0.1.0

https://github.com/FEBAutonomous/feb-racing
https://febautonomous.github.io/feb-racing/

Contents

1 Why you cannot just pull 2

2 Save your work 2

3 Set up release 2 2

4 Put your driver back 2

5 What changed for your code 2

6 Run it 3

7 Look inside 3

8 Score and submit 4

9 When something is wrong 4

1 Why you cannot just pull

Release 2 rewrote the repository's history. In an old clone, `git pull` stops with *refusing to merge unrelated histories*, and the old launcher `feb-sim` no longer exists. You clone once more, and carry your `stack/` folder across by hand. Nothing else in the old clone is yours. After this, `./feb update` covers every future release; you never do this again.

2 Save your work

Your code is in the old clone's `stack/` folder, and it is not on GitHub. Copy it out *before* anything else:

```
cp -r ~/feb-racing/stack ~/feb-stack-backup
ls ~/feb-stack-backup
```

You should see your package folders. If you changed anything outside `stack/`, copy that too. Then delete the old clone:

```
rm -rf ~/feb-racing
```

Windows: do all of this in your WSL 2 Ubuntu terminal.

3 Set up release 2

If Docker, VS Code and Foxglove are already installed, this is the whole setup:

```
mkdir -p ~/FEB && cd ~/FEB
git clone git@github.com:FEBAutonomous/feb-racing.git
cd feb-racing
./feb setup --name "Your Name"
```

It pulls the new image, downloads the simulator and creates your box, a persistent container with your whole `~/FEB` folder inside. Then `./feb up`: the simulator opens, arrow keys drive. Close the window to stop. That proves the new devkit works before your code goes in.

4 Put your driver back

```
cp -r ~/feb-stack-backup/* ~/FEB/feb-racing/stack/
ls ~/FEB/feb-racing/stack
```

Your packages are now in the new `stack/`, which is the workspace the box builds.

5 What changed for your code

Three things differ from release 1, and the first two matter for every driver.

1. The topics: the car's, not the simulator's

Release 1 drivers subscribed to the simulator directly, `/autodrive/roboracer_1/lidar` and published `/autodrive/roboracer_1/throttle_command`. Release 2 drivers talk the car's topics, and a house node (the *sim adapter*) translates them for the simulator. The same code then runs on the car unchanged.

Was	Now	Type
`/autodrive/roboracer_1/lidar`	`/scan`	sensor_msgs/LaserScan
`/autodrive/roboracer_1/imu`	`/imu`	sensor_msgs/Imu
encoders, ips	`/odom`	nav_msgs/Odometry (dead-reck
`/autodrive/roboracer_1/front_camera`	`/camera/color/image_raw`	sensor_msgs/Image
`throttle_command` + `steering_command`	`/drive`	ackermann_msgs/AckermannDr

On `/drive` you publish a **speed in m/s** and a **steering angle in radians**, left positive. No more throttle fractions. In the gap follower this is about thirty lines; the starter kit's `starter_kit/feb_driver/fe` shows the new shape.

2. The driver's card

Each driver package now carries a `driver.yaml` at its top level, so `feb up`, `feb practice` and the car know how to start it:

```
name: my_driver
kind: walls                 # walls | cones | any
maintainer: your name
packages: [my_driver]       # what feb submit copies: your own packages
modes:
  trackdrive: {launch: drive.launch.py, max_speed: 3.0}
  inspection: {launch: drive.launch.py, max_speed: 0.8}
default_mode: trackdrive
```

`launch` is a file in your package's `launch/` folder. If you had no launch file, copy `starter_kit/feb_driver/lau` and change the package and executable names.

3. Config files (optional)

Tuning numbers go in `config/driver.yaml`, loaded by the launch file; values that differ on the real car go in `config/driver.car.yaml`, loaded on top only on the car. Not required to run.

6 Run it

```
./feb up --driver my_driver
```

The box builds your package with `colcon` on the first run, then every run after that is incremental. Build errors print in this terminal. Until your driver has its card, you can still run it the raw way:

```
./feb up --stack "ros2 launch my_driver drive.launch.py"
```

7 Look inside

```
./feb shell             # a terminal in the box: ros2 topic list, ros2 topic echo /
    drive
./feb shell --gui       # the same, plus the box's desktop in a browser tab (localhost
    :6080)
```

rviz in the browser: with `./feb up` running, in a second terminal `./feb shell --gui` then `rviz2 &`. On the browser desktop: Fixed Frame `odom`, Add by topic `/scan` and `/odom`, LaserScan style Points, size 4; Reliability Best Effort if a topic stays empty. `./feb code` opens VS Code inside the box.

8 Score and submit

```
./feb practice --driver my_driver      # ten laps on each qualifying track
./feb submit                           # results to the scoreboard, code to feb-stack
```

A run counts only with zero collisions. Everything else, the dead-man, the car, is in `member_guide.pdf`.

9 When something is wrong

It says	Do
`no driver.yaml with name 'my_driver'`	no card with that `name:` under `stack/`; add it (section 5.2)
`refusing to merge unrelated histories`	you are in the old clone; follow this document from the top
nothing drives, `/drive` is silent	your node still publishes the old throttle topics; see "The topics"
build error mentioning `autodrive_msgs`	that package is gone; the adapter owns the simulator topics now
`this clone sits directly in /home/...`	`mkdir ~/FEB && mv ~/feb-racing ~/FEB/`
`can't copy '...resource/my_driver'`	you renamed the package but not the empty marker file in `resource/`; rename it to the package name